

Step 4 — Graceful Degradation When One MCP Server Is Down — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume Step 3's full team (`state.py` , `supervisor.py` , `notes_agent.py` , `web_agent.py` , `writer_agent.py` , `graph.py`) already works end to end.

Basic Version

Approach 1 — the direct way, a broad try/except and a string prefix check

```
# web_agent.py (relevant part)
import asyncio

WEB_UNAVAILABLE_PREFIX = "WEB_UNAVAILABLE"

def run_web_agent(task, timeout=10.0):
    try:
        return asyncio.run(asyncio.wait_for(_run_web_agent(task), timeout=timeout))
    except Exception as e:
        return f"{WEB_UNAVAILABLE_PREFIX}: {e}"
```

```
# web_agent.py (node function)
from langgraph.types import Command

def web_agent_node(state):
    result = run_web_agent(state["task"])
    visited = state.get("visited", []) + ["web_agent"]
    if result.startswith(WEB_UNAVAILABLE_PREFIX):
        print(f"[web_agent] server unreachable: {result}, continuing without it")
        return Command(goto="supervisor", update={"web_findings": "", "web_unavailable": ""})
    return Command(goto="supervisor", update={"web_findings": result, "visited": visited})
```

This catches the failure and stops the crash. It's missing the specific `asyncio.TimeoutError` distinction (bundling it into the same broad `except Exception` , which is fine here since this is the

one deliberate boundary where broad catching is correct) and doesn't yet update the Writer's prompt — that's what Intermediate adds.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a named timeout error case, and the flag reaching the Writer's prompt

```
# web_agent.py (full relevant file)
import asyncio
import json
from mcp_connection import connect_stdio_server
from openai import OpenAI
from tool_bridge import mcp_tools_to_openai_schema, call_mcp_tool
from langgraph.types import Command
from state import TeamState

client = OpenAI()
SYSTEM_PROMPT = "You are a helpful assistant that answers questions using the fet
WEB_UNAVAILABLE_PREFIX = "WEB_UNAVAILABLE"

async def _run_web_agent(task: str, max_iterations: int = 5) -> str:
    async with connect_stdio_server("python", ["web_server.py"]) as session:
        tools_result = await session.list_tools()
        openai_tools = mcp_tools_to_openai_schema(tools_result.tools)

        messages = [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": task},
        ]

        for _ in range(max_iterations):
            response = client.chat.completions.create(
                model="gpt-4o-mini", messages=messages, tools=openai_tools
            )
            reply = response.choices[0].message
            messages.append(reply)

            if not reply.tool_calls:
                return reply.content

            for call in reply.tool_calls:
                args = json.loads(call.function.arguments)
                result_text = await call_mcp_tool(session, call.function.name, ar
```

```

        messages.append({"role": "tool", "tool_call_id": call.id, "content": result})

    return "Reached max iterations without a final answer."

def run_web_agent(task: str, timeout: float = 10.0) -> str:
    try:
        return asyncio.run(asyncio.wait_for(_run_web_agent(task), timeout=timeout))
    except asyncio.TimeoutError:
        return f"{WEB_UNAVAILABLE_PREFIX}: timed out after {timeout}s connecting"
    except Exception as e:
        return f"{WEB_UNAVAILABLE_PREFIX}: {type(e).__name__}: {e}"

def web_agent_node(state: TeamState) -> Command:
    result = run_web_agent(state["task"])
    visited = state.get("visited", []) + ["web_agent"]

    if result.startswith(WEB_UNAVAILABLE_PREFIX):
        print(f"[web_agent] {result} -- continuing without it")
        return Command(
            goto="supervisor",
            update={"web_findings": "", "web_unavailable": True, "visited": visited}
        )

    return Command(goto="supervisor", update={"web_findings": result, "visited": visited})

```

```

# writer_agent.py (updated)
from openai import OpenAI
from langgraph.graph import END
from langgraph.types import Command
from state import TeamState

client = OpenAI()

WRITER_PROMPT = """You are writing a short status report from research findings.

Notes findings:
{notes_findings}

Web findings:
{web_findings}

Write one short, clear report. State what is known, citing which source(s)
support each point. Do not write one paragraph per source -- organize by
topic or conclusion instead.{degradation_note}"""

```

```
def run_writer(notes_findings: str, web_findings: str, web_unavailable: bool = False):
    degradation_note = ""
    if web_unavailable:
        degradation_note = (
            "\n\nNote: the web status page could not be reached this time. "
            "Say plainly in the report that the web check was not completed -- "
            "do not guess or invent what it might have said."
        )

    prompt = WRITER_PROMPT.format(
        notes_findings=notes_findings or "(not checked for this request)",
        web_findings=web_findings or "(not checked for this request)",
        degradation_note=degradation_note,
    )
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def writer_agent_node(state: TeamState) -> Command:
    report = run_writer(
        state.get("notes_findings", ""),
        state.get("web_findings", ""),
        state.get("web_unavailable", False),
    )
    return Command(goto=END, update={"report": report})
```

```
# state.py (final)
from typing import TypedDict

class TeamState(TypedDict):
    task: str
    needed: list[str]
    visited: list[str]
    notes_findings: str
    web_findings: str
    web_unavailable: bool
    report: str
```

```
# supervisor.py (unchanged in logic, "done" check only ever looked at visited)
from langgraph.graph import END
from langgraph.types import Command
from state import TeamState

def figure_out_needed(task: str) -> list[str]:
    needed = []
    task_lower = task.lower()
    if "note" in task_lower:
        needed.append("notes_agent")
    if "http" in task_lower or "fetch" in task_lower or "page" in task_lower:
        needed.append("web_agent")
    if not needed:
        needed.append("notes_agent")
    return needed

def supervisor_node(state: TeamState) -> Command:
    needed = state.get("needed") or figure_out_needed(state["task"])
    visited = state.get("visited") or []

    for specialist in needed:
        if specialist not in visited:
            return Command(goto=specialist, update={"needed": needed})

    return Command(goto="writer_agent")
```

Expected output, running task 3 with `web_server.py` never started:

```
[web_agent] WEB_UNAVAILABLE: timed out after 10.0s connecting to web_server.py --
Report:
Project Atlas is targeted for a Q3 launch, according to team notes, with
legal sign-off still pending. The web status page could not be checked
this time, so its current status could not be confirmed.
```

Difference from Basic: `supervisor_node`'s "done" check was never checking whether `web_findings` was non-empty in the first place — it only ever checked `visited` — so no change was actually needed there once `web_agent_node` correctly marks itself visited either way; this confirms the design decision from Hint 1's Advanced section was right. The real addition here is the flag traveling all the way into `run_writer`'s prompt, not stopping at `state["web_unavailable"]`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — `test_degraded_run.py` , both scenarios in one file

```
# test_degraded_run.py
from graph import build_graph

TASK = "Check my notes for anything about Project Atlas, and also fetch its statu

def run_once(label):
    app = build_graph()
    final_state = app.invoke({
        "task": TASK, "needed": [], "visited": [], "notes_findings": "",
        "web_findings": "", "web_unavailable": False, "report": "",
    })
    print(f"\n=== {label} ===")
    print("Visited:", final_state["visited"])
    print("web_unavailable:", final_state["web_unavailable"])
    print("Report:\n" + final_state["report"])
    return final_state

if __name__ == "__main__":
    # Run 1: with web_server.py already running in another process --
    # confirm the full, non-degraded report still works exactly as Step 3 left it
    print("Start web_server.py in another terminal now, then press Enter.")
    input()
    normal = run_once("Both servers up")
    assert normal["web_unavailable"] is False
    assert normal["web_findings"] != ""

    # Run 2: stop web_server.py (Ctrl+C it in the other terminal), then press Ent
    print("\nStop web_server.py now, then press Enter.")
    input()
    degraded = run_once("Web server down")
    assert degraded["web_unavailable"] is True
    assert degraded["notes_findings"] != ""
    assert "notes_agent" in degraded["visited"]
    assert "web_agent" in degraded["visited"]
```

```
print("\nAll degradation checks passed.")
```

Expected output (second run, abridged):

```
=== Web server down ===
Visited: ['notes_agent', 'web_agent']
web_unavailable: True
Report:
Project Atlas is targeted for a Q3 launch, according to team notes...
The web status page could not be checked this time.

All degradation checks passed.
```

Approach 2 — confirming the Notes Agent is genuinely unaffected

```
# test_notes_unaffected_by_web_outage.py
# Run this with web_server.py NOT running -- confirms the Notes Agent's
# own connection is completely isolated from the Web Agent's failure.
from notes_agent import run_notes_agent

result = run_notes_agent("What do my notes say about Project Atlas?")
print(result)
assert "Q3" in result or "Atlas" in result
print("Notes agent unaffected by web outage -- isolation confirmed.")
```

Expected output:

```
Your notes say Project Atlas is targeted for a Q3 launch...
Notes agent unaffected by web outage -- isolation confirmed.
```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate proves the degradation mechanism works in principle, read from code. Approach 1 is the real, end-to-end proof against your actual `web_server.py` process being started and stopped for real — not simulated in code — the same standard Project 8 Step 4 held its own fallback test to. Approach 2 checks something Approach 1 doesn't directly prove: that the Notes Agent's own success has nothing to do with the Web Agent's failure — genuinely separate MCP connections, not one shared connection with a workaround bolted on.

Which one should you actually write? Both, and keep Approach 1's manual up/down test as your standard "is this project actually done" check, not just something you ran once. It's slower than a fully automated test (it asks you to actually start and stop a process), but it's the only version that proves the real failure mode this step is about, the same way Project 8 Step 1's Advanced hint

insisted on watching a real broken connection with your own eyes before trusting any code that claims to handle it. Approach 2's isolation check is worth keeping as a fast regression test — if it ever starts failing, that's an early, clear signal that a later change accidentally coupled two specialists that were supposed to stay independent.